# Simple Shell

Lab 1 Report — Operating Systems

Yehia Said Gewily | ID: 21010705 | February 2026

## Overview

For this lab I wrote a basic Unix shell in C that supports the built-in commands `cd`, `echo`, `export`, and `exit`, along with variable expansion, running external programs, and background processes via `&`. When a background child finishes, the shell logs it to a file. Everything lives in one `.c` file and compiles cleanly with `gcc`.

## Main Functions

### fork()

`fork()` is how Unix creates a new process. It makes an exact copy of the current process and both copies continue running from that same point. They're identical except for what `fork()` returns to each of them.

**Return values:**

- **negative** — the fork failed (e.g. system is out of resources). Handle the error and stop.
- **0** — we're inside the *child* process. This is where we call `execvp()`.
- **positive** — we're in the *parent*, and the value is the child's PID. We use it to wait for the child if it's a foreground command.

```
pid_t pid = fork();
if (pid < 0)  { perror("fork failed"); return; }
if (pid == 0) { execvp(argv[0], argv); exit(EXIT_FAILURE); }
// parent continues here, pid = child's PID
```

### execvp()

After `fork()`, the child is still just a copy of the shell. `execvp()` replaces the child's entire memory image with a new program like `ls` or `grep`. The `v` means we pass a `NULL`-terminated argument array, and the `p` means it searches `PATH` automatically so we don't need full paths.

If it succeeds it **never returns** — the child is now a completely different program. If it does come back, something went wrong, so we print an error and exit.

```
execvp(argv[0], argv);
// only here on failure:
fprintf(stderr, "%s: command not found\n", argv[0]);
exit(EXIT_FAILURE);
```

### waitpid()

`waitpid()` makes the parent pause until a child finishes. For foreground commands I block on the child's PID. For the signal handler I use the `WNOHANG` flag so it returns immediately instead of blocking if no child has exited yet — this is important so the shell doesn't freeze.

### sigaction() and SIGCHLD

When a background child finishes, the kernel sends the parent a `SIGCHLD` signal. I register a handler for it using `sigaction()` which is more reliable than the older `signal()` because it gives us the `SA_RESTART` flag. That flag tells the kernel to automatically restart any system calls that get interrupted mid-way by the signal, so things like `scanf` don't randomly fail.

### Other notable functions

- `chdir()` — changes the shell's working directory for the built-in `cd`.
- `setenv()` — exposes exported variables to child processes via the environment.
- `getcwd()` — fetches the current directory to display in the prompt.
- `strdup()` / `strtok()` — used in the tokeniser to split input into heap-allocated tokens we can safely pass to `execvp()`.

## Why Use a Subprocess?

> If we ran external commands directly inside the shell process, calling `execvp()` would *replace the shell itself* with the new program — and the shell would be gone. By forking first, the child takes the hit and the parent lives on. A crash or infinite loop in the child has zero effect on the shell. That's the whole point of the parent/child process model.

## Zombie Processes

### What is a zombie?

A zombie is a process that has **finished executing** but whose entry in the process table hasn't been cleaned up yet. The kernel keeps that small record around, waiting for the parent to call

`wait()` and collect the exit status. It's not using CPU or memory, but it still occupies a slot in the process table.

In our shell, zombies show up with background jobs. We launch the child and immediately return to the prompt without waiting, so when the child exits, its record just sits there uncollected.

### How I handled them

I registered a `SIGCHLD` handler. Every time any child exits, the kernel automatically sends us that signal. Inside the handler I loop with `WNOHANG` until all finished children are reaped:

```
static void on_child_exit(int sig) {
    (void)sig;
    while (waitpid(-1, NULL, WNOHANG) > 0) {}
    FILE *log = fopen(LOG_FILE, "a");
    if (log) { fprintf(log, "Child process was terminated\n"); fclose
        (log); }
}
```

The loop is necessary because multiple children can exit at nearly the same time and signals don't queue up — we could receive a single `SIGCHLD` for several children, so we keep calling `waitpid` until it has nothing left to reap.

## Interesting Things I Found

**Variable expansion inside quotes.** I made sure `$VAR` gets expanded even when the argument is in double quotes, matching how real bash works. The `expand_token()` function handles both cases.

**Word splitting after expansion.** If a variable expands to something containing spaces (e.g. `ARGS="-l -a"` then `ls $ARGS`), I split it back into separate tokens with `strtok()` after expanding. This is closer to how a real shell handles it.

**Memory management.** The tokeniser uses `strdup()` for each token, so I had to make sure `free_argv()` runs on every path through the main loop, including early exits, otherwise there's a memory leak on every single command.

**scanf and EOF.** Using `scanf(" %[^\n]", input)` reads full lines and skips leftover newlines. Checking the return value against `EOF` handles Ctrl+D cleanly so the shell exits gracefully instead of spinning in an infinite loop.